SESSION1

From Pixels to Visual Features

Image Processing • Edge Detection • Feature Extraction

Pixels → Filtering → Edges → Corners → Keypoints → Descriptors → Matching

About Me

Current Role & Domain

Role: Principal AI / CV Engineer

Domain: ADAS, Autonomous Driving & Deep Perception Systems

Background & Research

Education: Master's in Smart Mobility (AI for ADAS, UAV and Robots)

Focus: Multi-Modal Realtime Perception

Let's Calibrate

Quick show of hands — no wrong answers, just helping gauge background knowledge:

OpenCV Experience

Image loading, color conversions, filtering

ML / Deep Learning

Basic regression, classification, loss functions

Python + NumPy

Array slicing, vectorization, matrix ops

Course Arc

S1–2

Image Processing & Features

Filtering, gradients, SIFT/ORB, classical CV

S3–5

CNNs & Deep Reps

Convolutions, backprop, feature maps

S7–9

Object Detection

Two-stage vs One-stage (YOLO, Faster R-CNN)

S10

Segmentation

Semantic & instance segmentation (U-Net, Mask R-CNN)

S11–12

Geometry & 3D

Stereo vision, epipolar geometry, depth

S13–15

ViTs & Foundations

Vision Transformers, CLIP, Foundation models

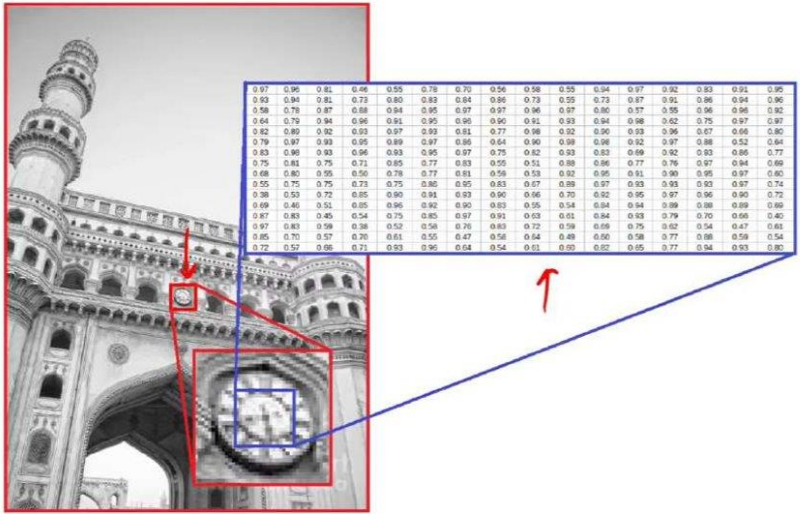
Goal: Understand a CV problem → Choose approach → Implement → Evaluate → Know limitations

Evaluation Criteria

20%	20%	25%	35%
Quiz 1	Quiz 2	Projects	End Semester
After Session 1 topics	After Session 2 topics	Best 3 of 4 • Kaggle-style	Implementation + Theory

* Implementation & deep understanding expected — not just memorizing formulas.

Image as a Matrix

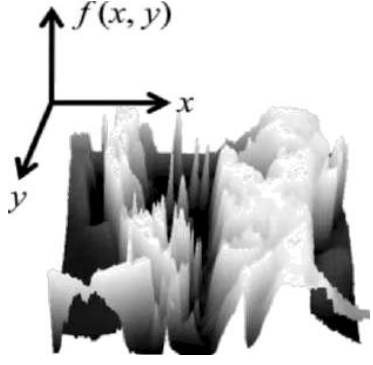


- Common to use one byte per value:
  0 = black, 255 = white
- One such matrix for every channel in color images

Image as a Matrix

- We can think of a (grayscale) image as a function f: $\mathbb{R}^2 \rightarrow \mathbb{R}$, giving the intensity at position (x,y)
- A digital image is a discrete (sampled, quantized) version of this function





Credit: Noah Snavely, Cornell Univ

Image Transformations

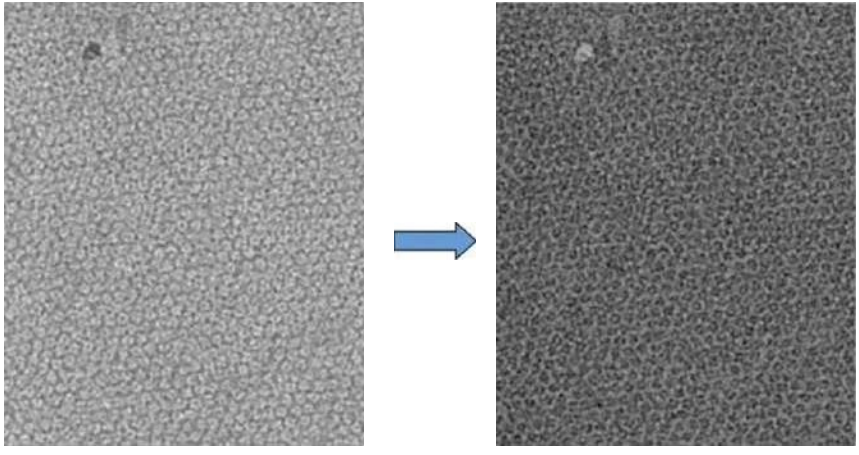


$$\hat{I}(x,y) = I(x,y) + 20$$

$$\hat{I}(x,y) = I(-x,y)$$

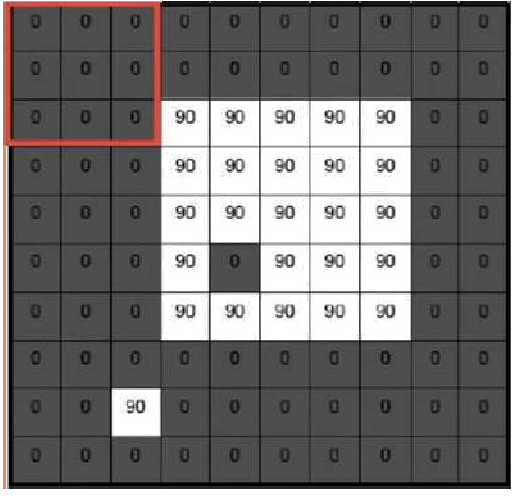
Point Operations: Example

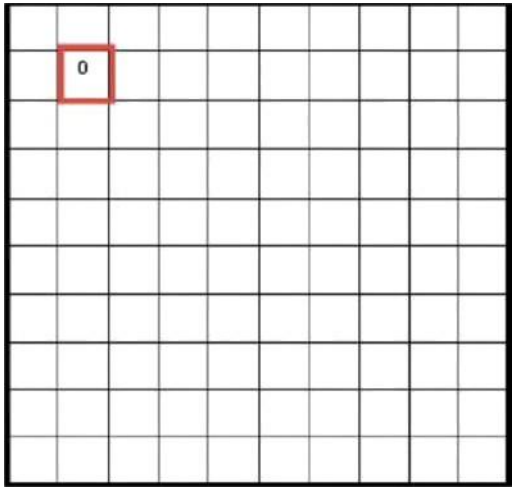
- Image Enhancement: Reversing the contrast
- How?



$$\hat{I}(m_0, n_0) = I_{MAX} - I(m_0, n_0) + I_{MIN}$$

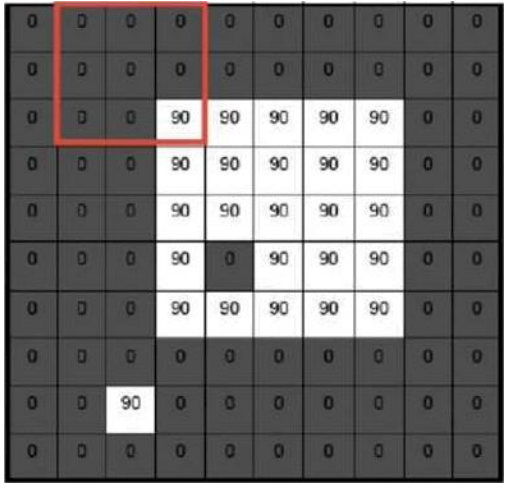
Local Operation Examples: Moving Average

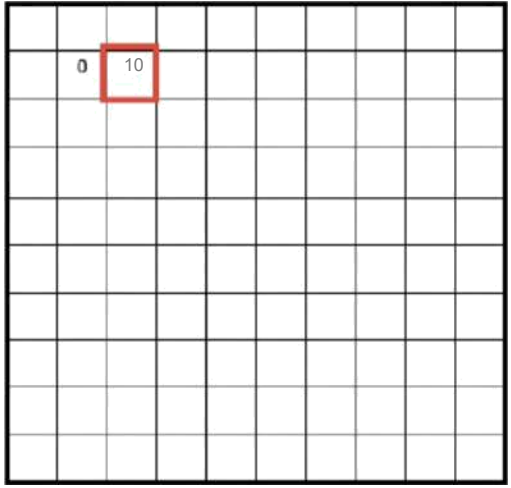




Credit: Steve Seítz, Univ of Washington

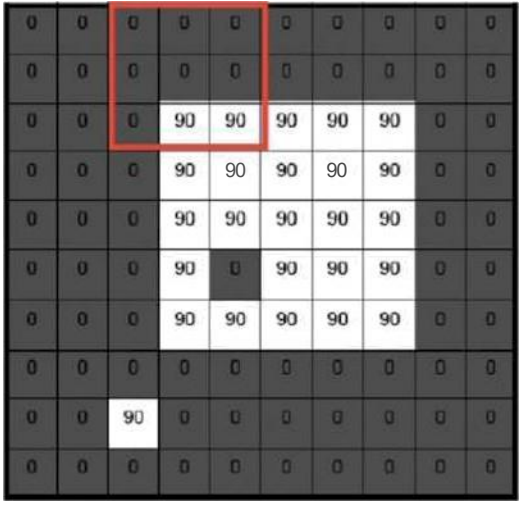
Local Operation Examples: Moving Average

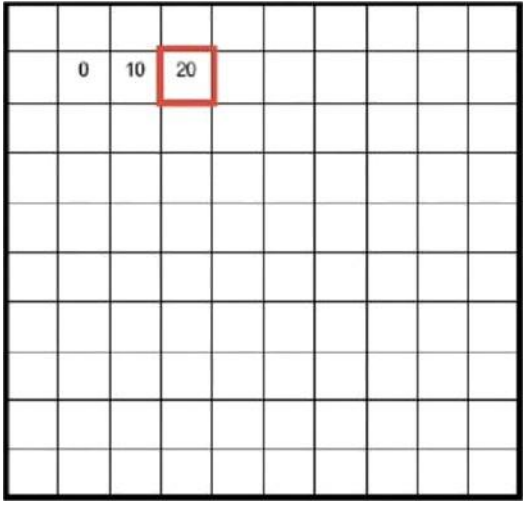




Credit: SteVe Seitz, Univ of Washington

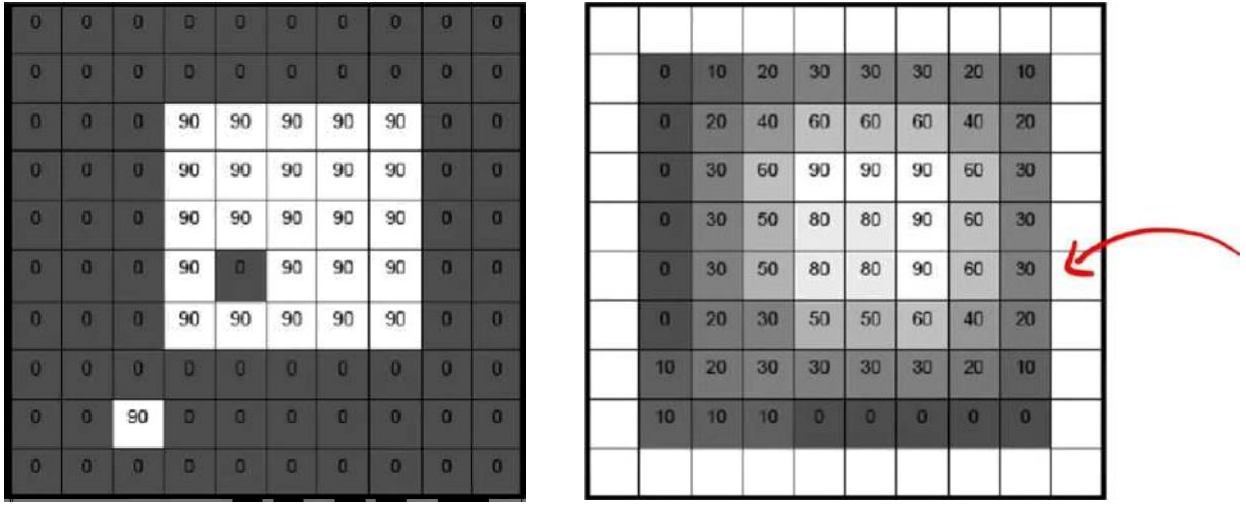
Local Operation Examples: Moving Average





Credit: Steve Seitz, Univ of Washington

Local Operation Examples: Moving Average



Credit: Steve Seitz, Univ of Washington

Image Processing Operations

- **Point Operations**
  - Output value at (m_o, n_o) is dependent only on the input value at the same coordinate
  - Complexity/pixel: Constant
- **Local Operations**
  - Output value at (m_o, n_o) is dependent on input values in a $p \times p$ neighborhood of that same coordinate
  - Complexity/pixel: $p2$
- **Global Operations**
  - Output value at (m_o, n_o) is dependent on all the values in the input $N \times N$ image
  - Complexity/pixel: N^2

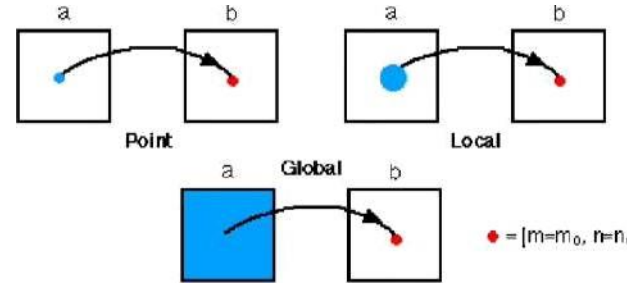
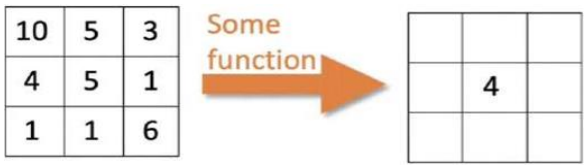


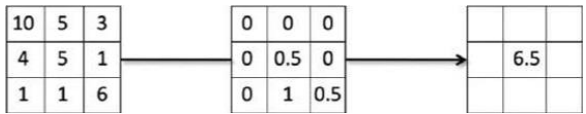
Image Filters: Linear Filter

Image Filter: Modify image pixels based on some function of a local neighborhood of each pixel



What's the function?

- Linear Filter: Replace each pixel by linear combination (a weighted sum) of neighbours
- Linear combination called kernel, mask or filter



Local image data | Kernel | Modified image data

Linear Filter: Cross-Correlation

Given a kernel of size (2k + 1) × (2k + 1):

Correlation defined as:

$$G(i,j) = \underbrace{\frac{1}{(2k+1)^2}}_{\text{Uniform weight to each pixel}} \underbrace{\sum_{u=-k}^{k} \sum_{v=-k}^{k} I(i+u, j+v)}_{\text{Loop over pixels in considered neighbourhood around } I(i,j)}$$

Linear Filter: Cross-Correlation

Given a kernel of size (2k + 1) × (2k + 1):

Correlation defined as:

$$G(i,j) = \underbrace{\frac{1}{(2k+1)^2}}_{\text{Uniform weight to each pixel}} \underbrace{\sum_{u=-k}^{k}\sum_{v=-k}^{k} I(i+u, j+v)}_{\text{Loop over pixels in considered neighbourhood around } I(i,j)}$$

Correlation defined as:

$$G(i,j) = \sum_{u=-k}^{k}\sum_{v=-k}^{k} \underbrace{H(u,v)}_{\text{Non-uniform weights}} I(i+u, j+v)$$

Linear Filter: Cross-Correlation

Given a kernel of size (2k + 1) × (2k + 1):

Correlation defined as:

$$G(i,j) = \underbrace{\frac{1}{(2k+1)^2}}_{\text{Uniform weight to each pixel}} \underbrace{\sum_{u=-k}^{k}\sum_{v=-k}^{k} I(i+u, j+v)}_{\text{Loop over pixels in considered neighbourhood around } I(i,j)}$$

Correlation defined as:

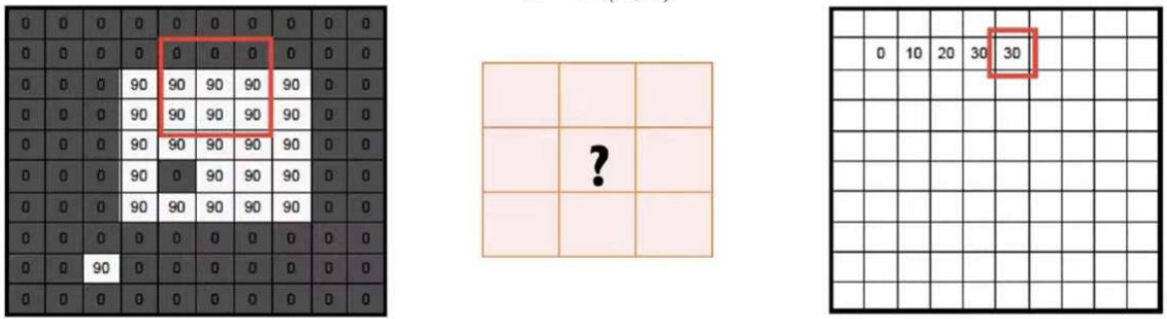
$$G(i,j) = \sum_{u=-k}^{k}\sum_{v=-k}^{k} \underbrace{H(u,v)}_{\text{Non-uniform weights}} I(i+u, j+v)$$

- Cross-correlation denoted by $G = H \otimes I$
- Can be viewed as "dot product" between local neighbourhood and kernel for each pixel
- Entries of kernel or mask $H(u,v)$ called filter co-efficients

Moving Average: Linear Filter

What values belong in the kernel H for the moving average example we saw earlier?

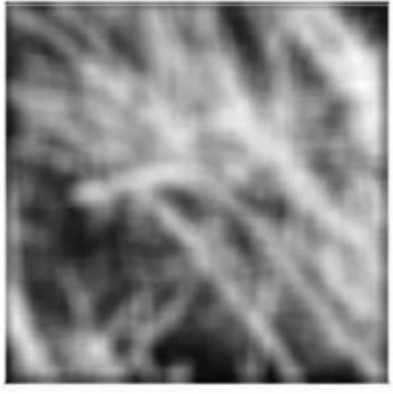
$I(i,j)$ $\otimes$ $H(u,v)$ $=$ $G(i,j)$



Moving Average Filter Example

Effect of moving average filter (also known as **box filter**):

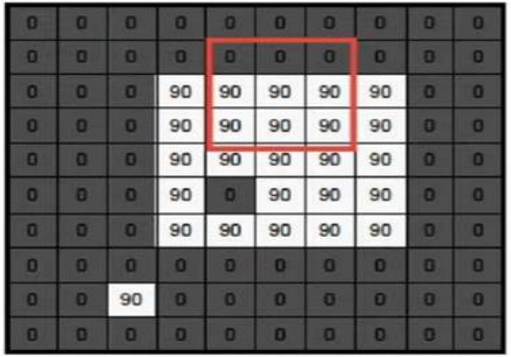




Gaussian Average Filter

What if we want nearest neighboring pixels to have the most influence on the output?

$I(i,j)$



$\otimes \quad H(u,v)$



Gaussian Average Filter

What if we want nearest neighboring pixels to have the most influence on the output?

$I(i,j)$

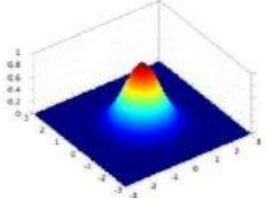
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$\otimes \quad H(u,v)$

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

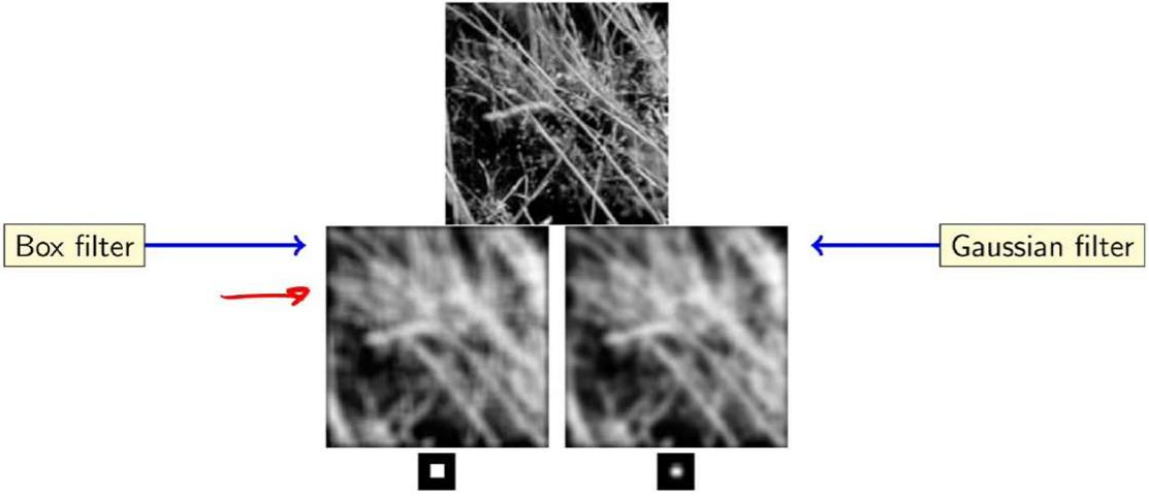
This kernel is an approximation of a 2D Gaussian function:

$$h(u,v) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{u^2+v^2}{\sigma^2}}$$



Credit: K Grauman, Univ of Texas Austin

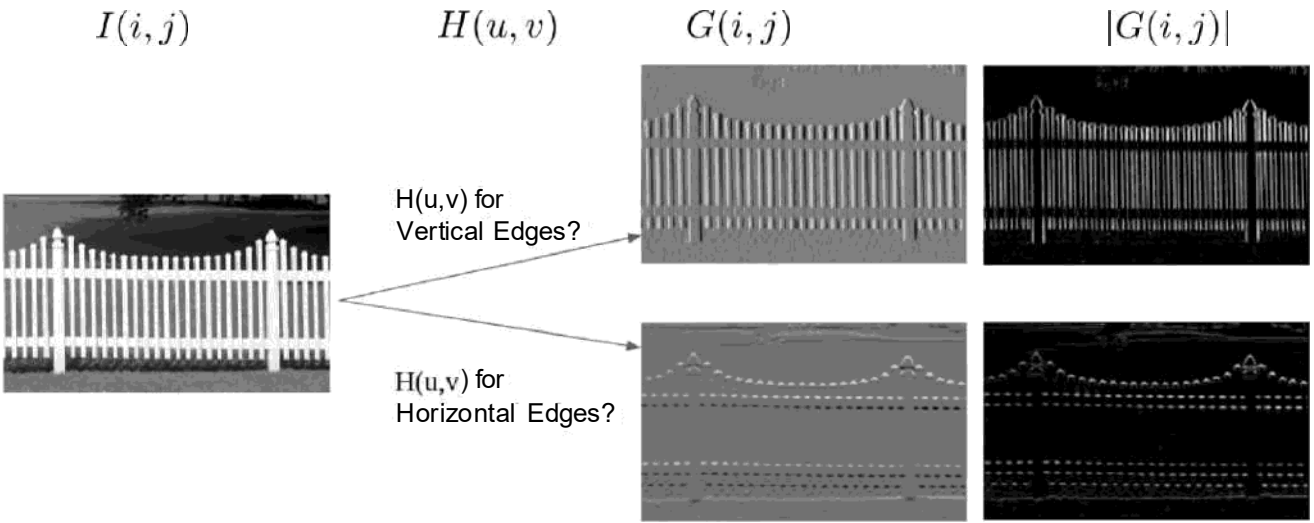
Averaging Filters: A Comparison



Credit: Steve Seitz, Univ of Washington

Other Filters: The Edge Filter

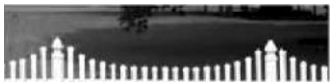
What should H look like to find the edges in a given image?



Other Filters: The Edge Filter

What should H look like to find the edges in a given image?

$I(i,j)$



$H(u,v)$

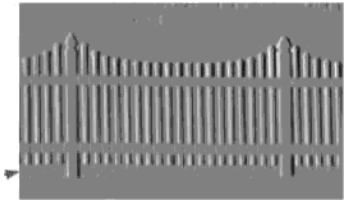
1/9

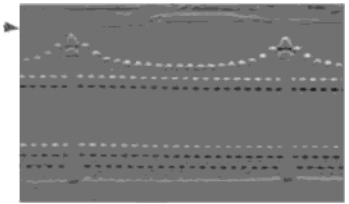
-1	0	1
-1	0	1
-1	0	1

1/9

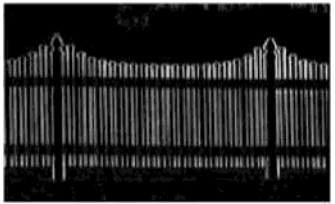
-1	-1	-1
0	0	0
1	1	1

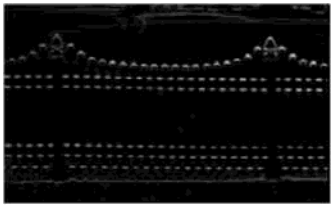
$G(i,j)$





$|G(i,j)|$





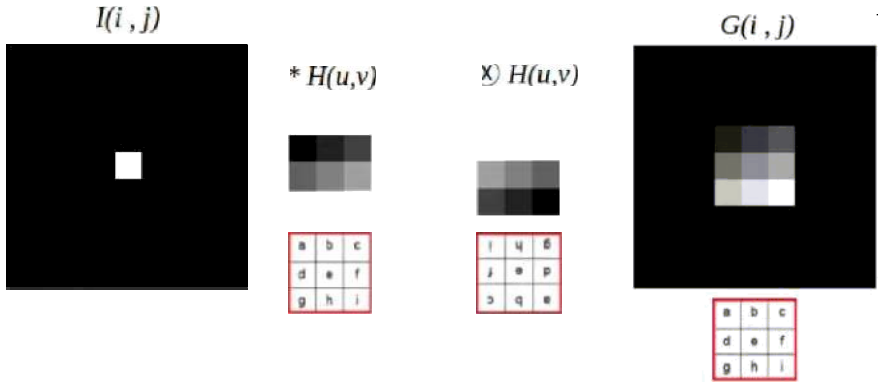
Introducing Convolution

Given a kernel of size (2k + 1) x (2k + 1):

Convolution defined as:

$$G(i,j) \sum_{u=-k}^{k} \sum_{v=-k}^{k} H(u,v)I(i-u,j-v)$$

- Equivalent to flip the filter in both directions (bottom to top, right to left) and apply cross-correlation
- Denoted by G = H x I



Properties of Convolution

Commutative: $a*b = b*a$

- Conceptually no difference between filter and signal

Associative : $a* (b*c) = (a*b)*c$

- We often apply filters one after the other: (((a * b1) * b2) * b3)
- This is equivalent to applying one cumulative filter: a * (b1 * b2 * b3)

Distributive over addition: $a*(b+c) = (a*b) + (a*c)$

- We can combine the responses of a signal over two or more filters by combining the filters

Scalars factor out: $ka * b = a * kb = k(a * b)$

Identity: Unit impulse e = [..., 0, 0, 1, 0, 0,...], a * e = a

Practical Issues

Ideal size for the filter?

The bigger the mask:

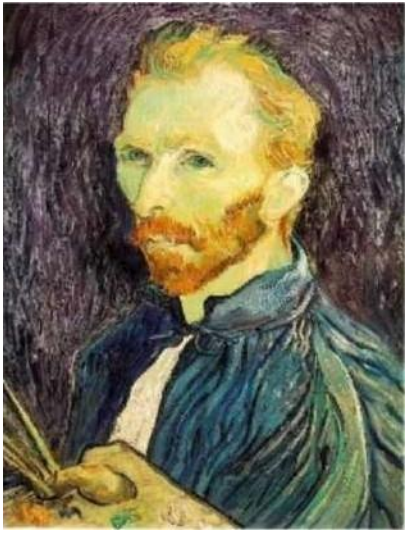
- more neighbours contribute
- smaller noise variance of output
- bigger noise spread
- more blurring
- more expensive to compute

What about the boundaries? Do we lose information?

- Without padding, we lose out on information at the boundaries.
- We can use a variety of strategies such as zero padding, wrapping around, copy the edge

Sub-sampling

Throw away every other row and column to create a ½ size image.





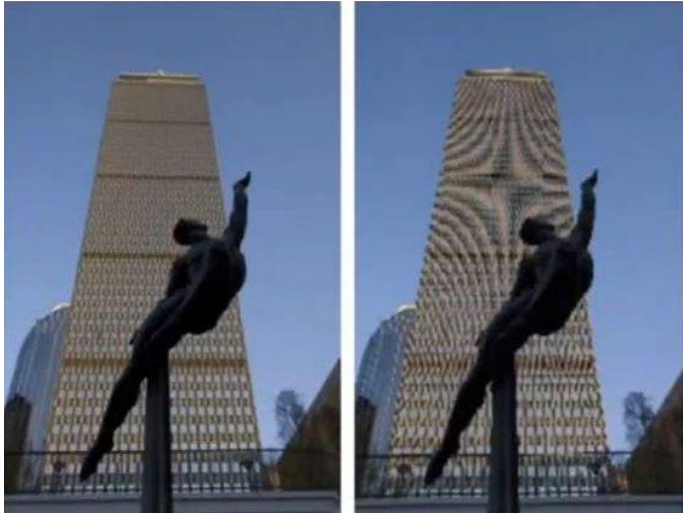
1/4



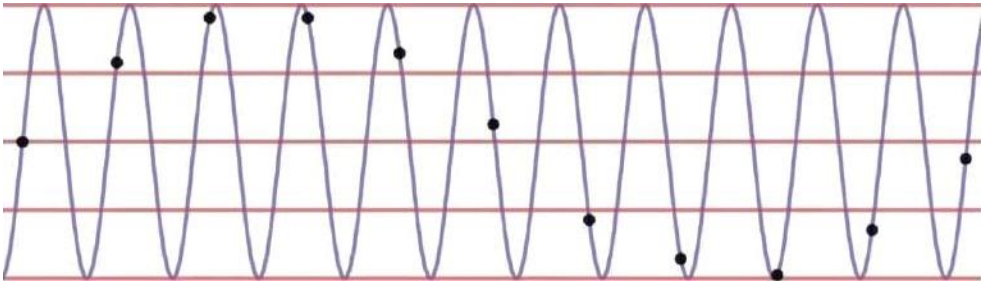
1/8

Sub-sampling

What's happening?

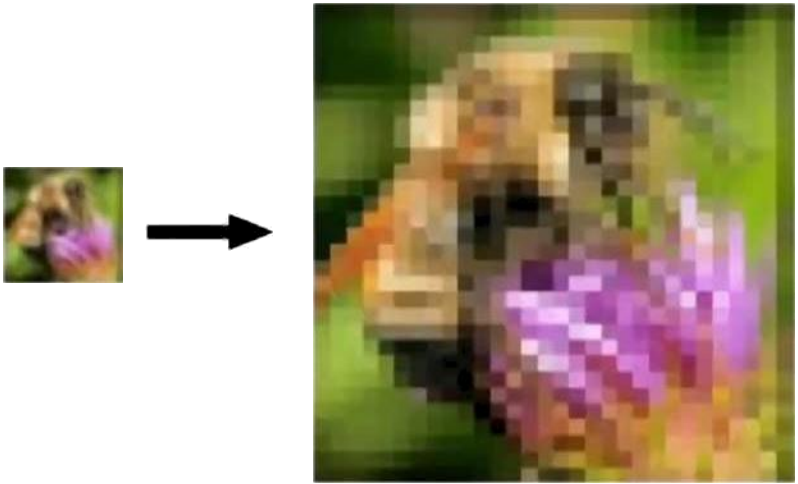


Aliasing



- Occurs when your sampling rate is not high enough to capture the amount of detail in your image.
- To do sampling right, need to understand the structure of your signal/image.
- The minimum sampling rate is called the Nyquist rate.

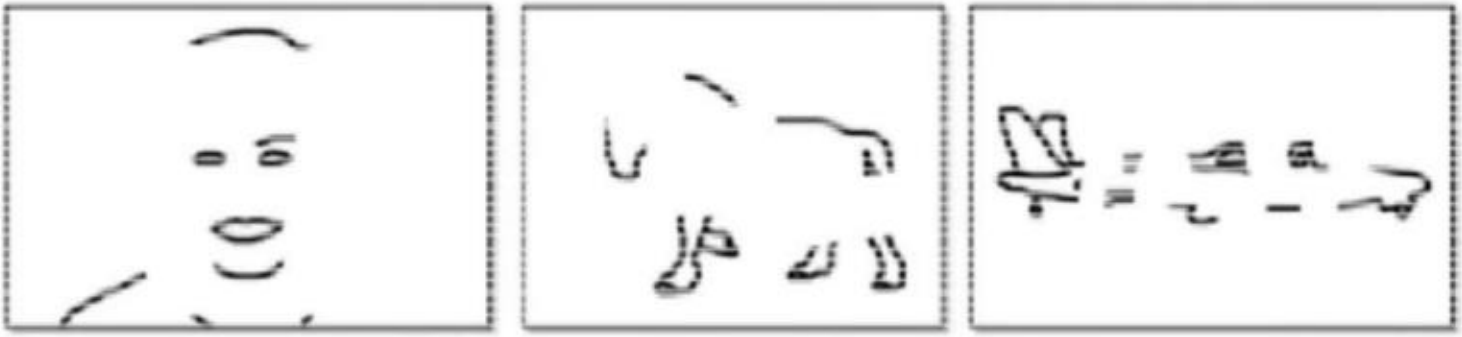
Up-sampling



10x

How to go from left to right? Interpolation. Simple method: Repeat each row and column 10 times (Nearest Neighbour Interpolation).

Edge Detection



- Map image from 2D matrix of pixels to a set of curves or line segments or contours
  ===> More compact representation than pixels
- Key idea7

Why are Edges Important?

- Group pixels into objects or parts
- Allow us to track important features (e.g., corners, curves, lines).
- Cues for 3D shape
- Guiding interactive image editing

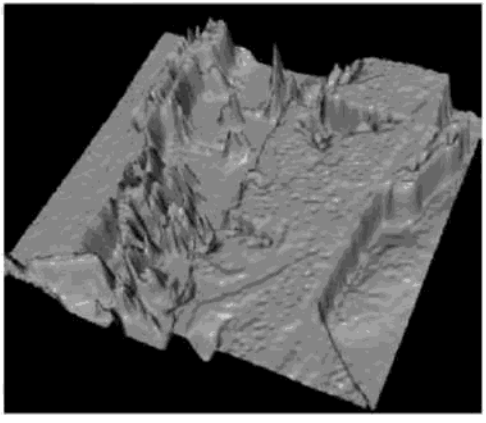


Source: Derek Hoiem

Edges in Images as Functions

Edges look like steep cliffs

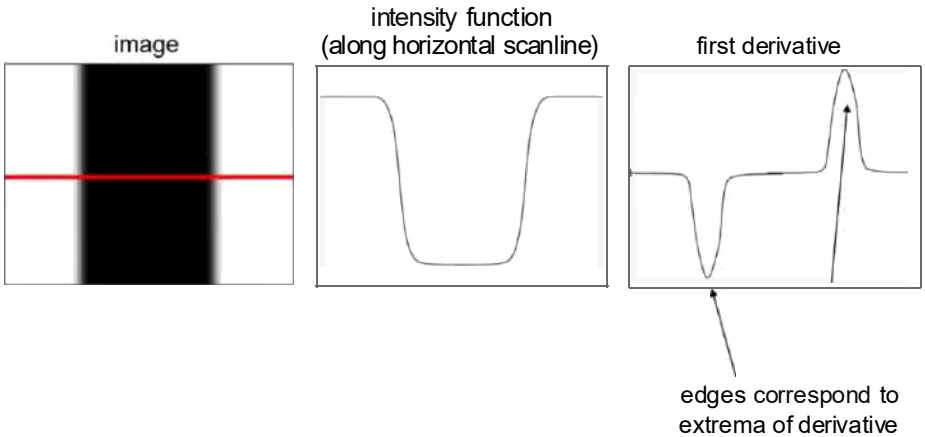


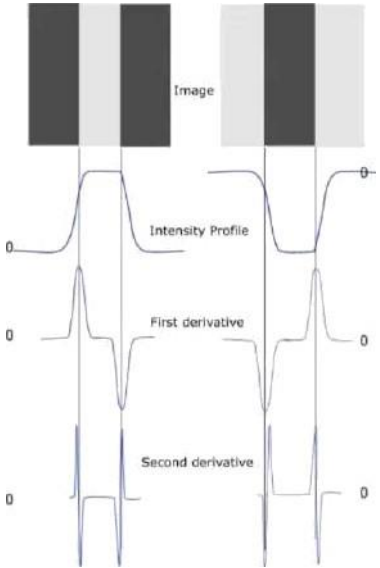


Source: N Snavely, R Urtasun

Derivatives and Edges

An edge is a place of rapid change in the image intensity function





Sources L Lazebnik, K Grauman and https://mipav.cit.Oih.gov/

Derivatives with Convolution

- For 2D function, $f(x, y)$, the partial derivative is:

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon, y) - f(x,y)}{\varepsilon}$$

- For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1,y) - f(x,y)}{1}$$

- To implement above as convolution, what would be the associated filter?

$\frac{\partial f(x,y)}{\partial x}$

-1	1







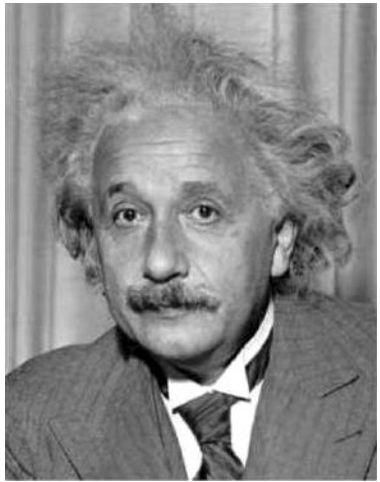
$\frac{\partial f(x,y)}{\partial y}$

-1
1

or

1
-1

Sobel Edge Detection Filters



1	0	-1
2	0	-2
1	0	-1

Sobel



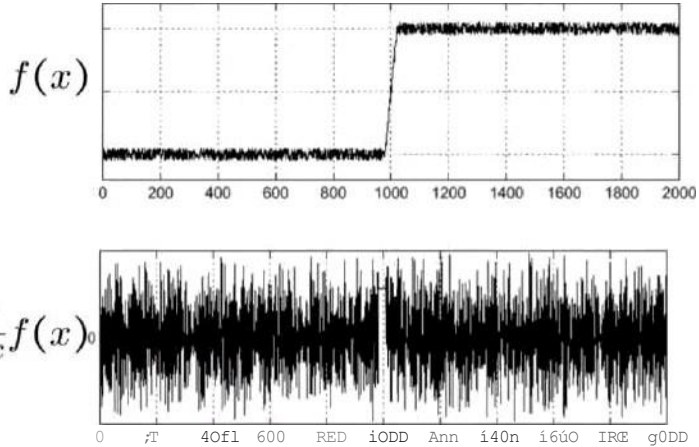
Vertical Edge
(absolute value)

Source: J Hays

Effect of Noise



Noisy input image

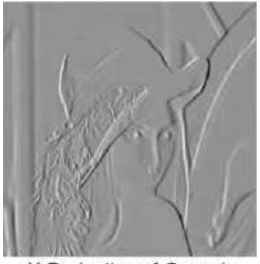


Now, where is the edge?

Source: S Seitz, K Grauman

Compute of Gradients





X-Derivative of Gaussian



Y-Derivative of Gaussian



Gradient Magnitude

Source: S Seitz, R Urtasun

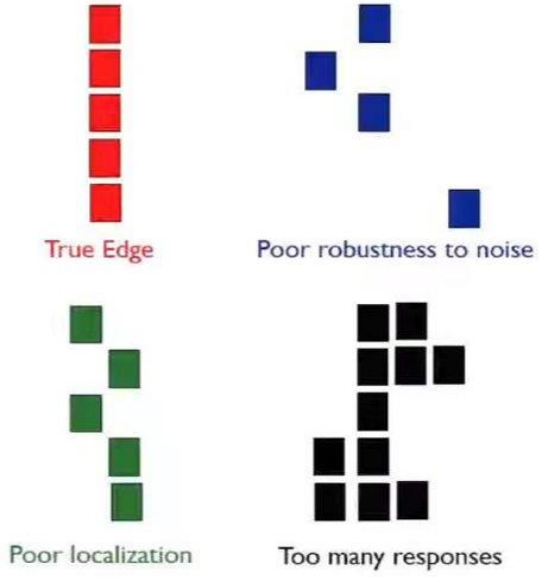
Properties of an Edge Detector

Criteria for a good edge detector?

Good detection: find all real edges, ignoring noise or other artifacts

Good localization: detect edges as close as possible to true edges

Single response: return one point only for each true edge point

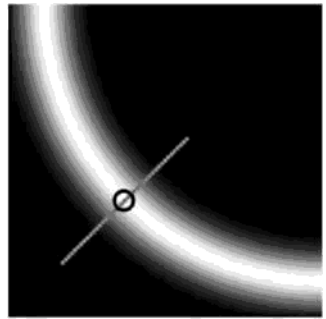


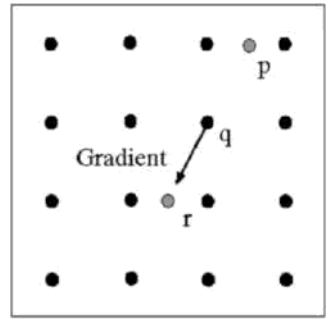
Source:

Non-Maxima Suppression



where is the edged





Check if pixel is local maximum along gradient direction:

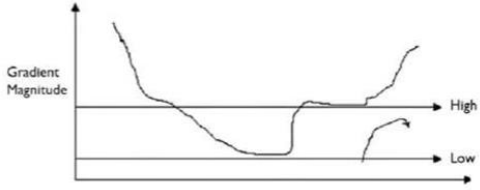
Could require checking interpolated pixels *P* and *r*

Source: N Snavely, R Urtasun

Hysteresis Thresholding

- Check for well-connected edges. How?
  - Use **hysteresis**: use a high threshold to start edge curves and a low threshold to continue them.

- How does it work?
  - If gradient at pixel > ‘High’ ⇒ ‘**edge pixel**’
  - If gradient at pixel < ‘Low’ ⇒ ‘**non-edge pixel**’
  - If gradient at pixel ≥ ‘Low’ and ≤ ‘High’ ⇒ ‘**edge pixel**’ iff it is connected to an ‘edge pixel’ directly or via pixels between ‘Low’ and ‘High’





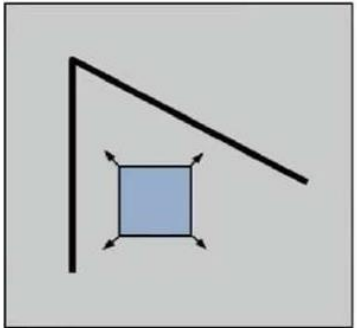
Canny Edge Detector

- Probably the most widely used edge detector in computer vision (J Canny 1986 [1])
- Algorithm:
  1. Filter image with derivative of Gaussian
  2. Find magnitude and orientation of gradient
  3. Non-maximum suppression
  4. Linking and thresholding (hysteresis):
     - Define two thresholds: low and high
     - Use the high threshold to start edge curves and the low threshold to continue them

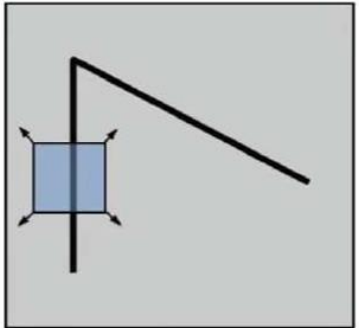
Source: D. Lowe, L. Fei-Fei, R Urtasun

From Blobs to Corners

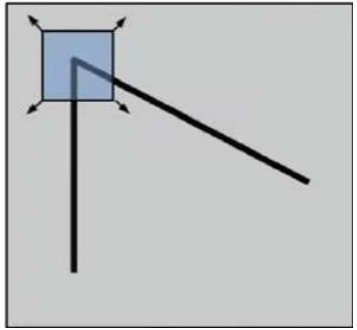
Consider a small window of pixels. How does the window change when you shift it?



"**flat**" region:
no change in all directions



"**edge**":
no change along the edge direction



"**corner**":
significant change in all directions

Credit: S Seitz, D Frolova, D Simakova, R Urtasun

Harris Corner Detector

- Compute gradients at each point in the image.
- Compute **A** for each image window to get its cornerness scores.
- Compute the eigenvalues/compute the following function M_c

$$M_c = \lambda_1\lambda_2 - \kappa(\lambda_1 + \lambda_2)^2 = det(A) - \kappa trace^2(A)$$

- Find points whose surrounding window gave larger cornerness response (M_c >threshold)
- Take points of local maxima, perform non-maximum suppression.

Autocorrelation

- In the previous slide, how to quantify the ”significant” change of the window?
- Answer: Autocorrelation function. Compute the sum of squared differences between pixel intensities with respect to small variations in the image patch position.

$$E_{AC}(\Delta \mathbf{u}) = \sum_{x,y} w(\mathbf{p}_i)[I(\mathbf{p}_i + \delta u) - I(\mathbf{p}_i)]^2$$

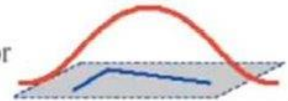
where $\mathbf{p}_i = (x, y)$, a particular position on the image.

Window function $w(x,y)$ =

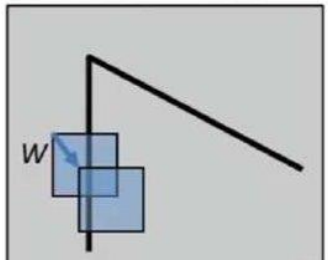


1 in window, 0 outside

or



Gaussian



Computing Autocorrelation

- The autocorrelation is $E_{AC}(\Delta \mathbf{u}) = \Delta \mathbf{u}^T \mathbf{A} \Delta \mathbf{u}$, with

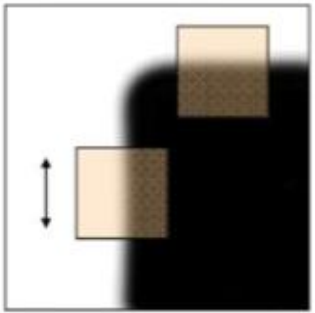
$$\mathbf{A} = \sum_u \sum_v w(u,v) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- The weighted summations have been replaced with discrete convolutions with the weighting kernel w.
- The eigenvalues of $\mathbf{A}$ reveal the amount of intensity change in the two principal orthogonal gradient directions in the window.

$$\mathbf{A} = \mathbf{U} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \mathbf{U}^T \qquad \text{with} \qquad \mathbf{A}\mathbf{u}_i = \lambda \mathbf{u}_i$$

Credit.' R Urtasun

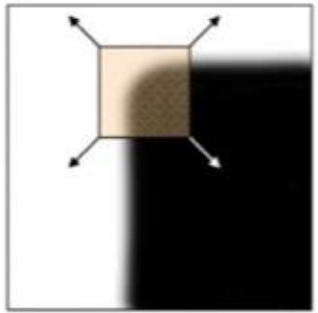
Computing Autocorrelation



"edge":

$\lambda_1 >> \lambda_2$

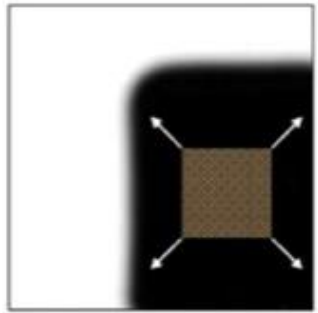
$\lambda_2 >> \lambda_1$



"corner":

λ_1 and λ_2 are large,

$\lambda_1 \sim \lambda_2$;



"flat" region

λ_1 and λ_2 are small;

Credit. K Cirauman, R Urtasun

Harris Corner Detector

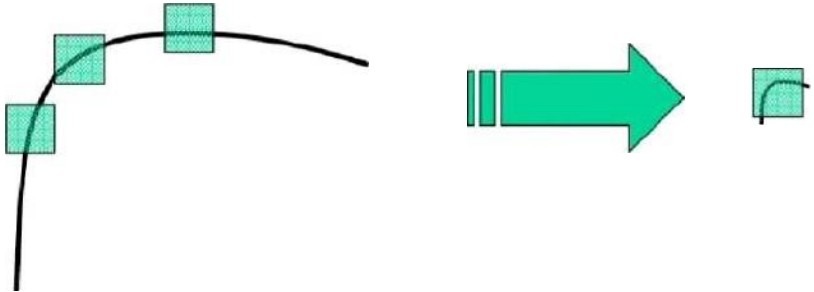
- Compute gradients at each point in the image.
- Compute **A** for each image window to get its cornerness scores.
- Compute the eigenvalues/compute the following function M_c

$$M_c = \lambda_1\lambda_2 - \kappa(\lambda_1 + \lambda_2)^2 = det(A) - \kappa trace^2(A)$$

- Find points whose surrounding window gave larger cornerness response (M_c >threshold)
- Take points of local maxima, perform non-maximum suppression.

Harris Corner Detector : Properties

Scale-invariant?



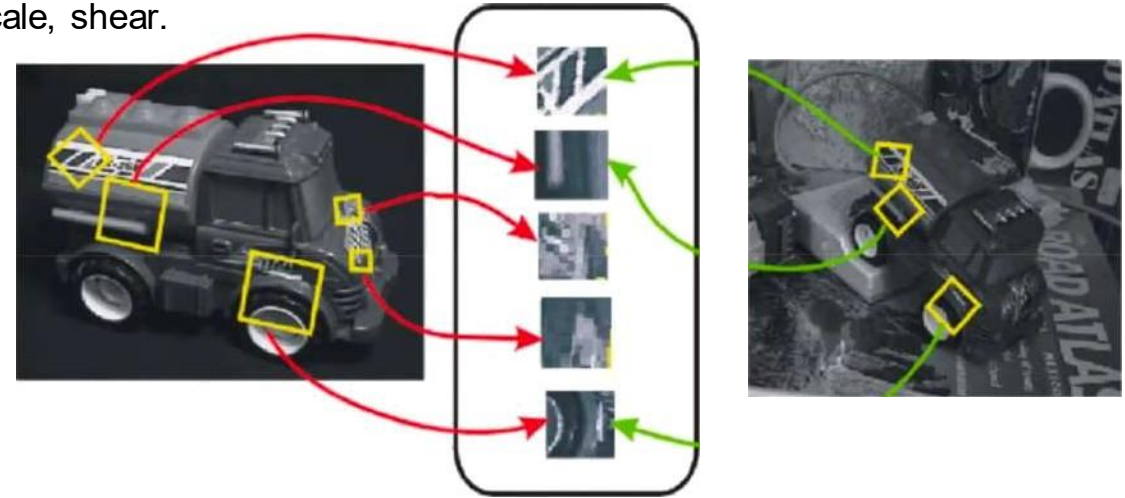
All points will be classified as edges

Corner !

Credit. R Urtasun

SIFT: Invariant Local Features

Image content is transformed into local feature coordinates that are invariant to translation, rotation, scale, shear.



SIFT Features

SIFT

Step 1: Scale-space Extrema Detection —Detect interesting points (invariant to scale and orientation) using DOG.

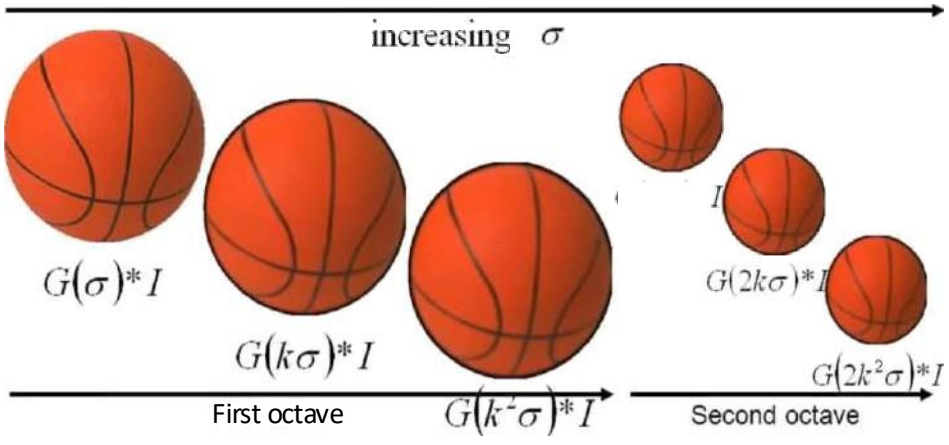
Step 2: Keypoint Localization - Determine location and scale at each candidate location, and select them based on stability.

Step 3: Orientation Estimation Use local image gradients to assign orientation to each localized keypoint. Preserve orientation, scale and location for each feature.

Step 4: Keypoint Descriptor - Extract local image gradients at selected scale around keypoint and form a representation invariant to local shape and illumination distortion.

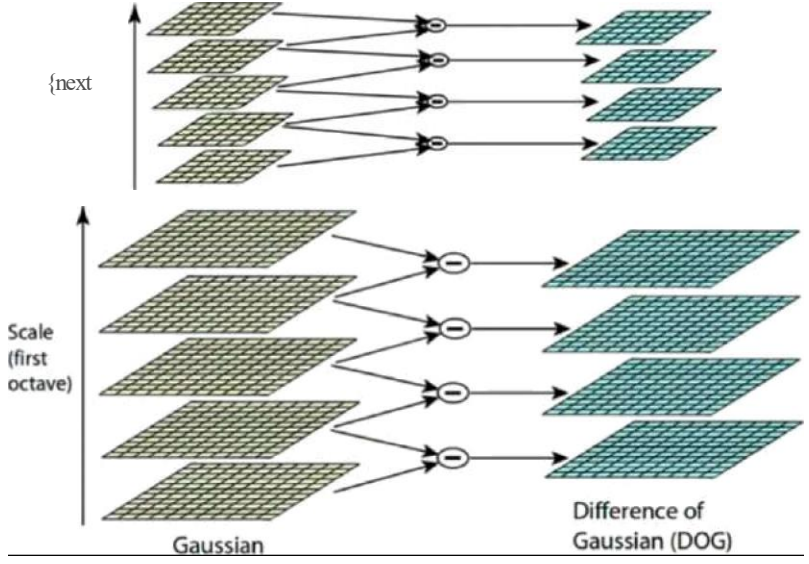
SIFT: Scale-space Extrema Detection

Constructing Scale Space



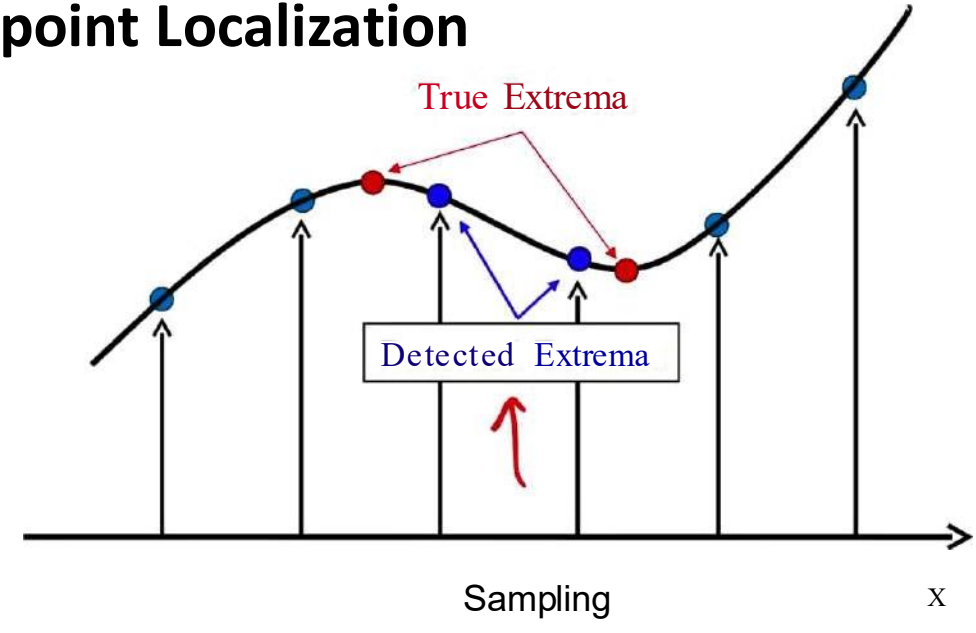
SIFT: Scale-space Extrema Detection

Difference of Gaussians



SIFT: Keypoint Localization

The Problem:



SIFT: Keypoint Localization

The Solution:

- Use Taylor series expansion of the scale-space function:

$$D(\mathbf{s}_0 + \Delta\mathbf{s}) = D(\mathbf{s}_0) + \frac{\partial D}{\partial \mathbf{s}}^T \Big|_{\mathbf{s}_0} \Delta\mathbf{s} + \frac{1}{2}\Delta\mathbf{s}^T \frac{\partial^2 D}{\partial \mathbf{s}^2}\Big|_{\mathbf{s}_0} \Delta\mathbf{s}$$

  where $\mathbf{s}_0 = (x_0, y_0, \sigma_0)^T$ and $\Delta\mathbf{s} = (\delta x, \delta y, \delta\sigma)^T$

- The location of the extremum, $\hat{\mathbf{s}}$, is determined by taking the derivative of this function with respect to $\mathbf{s}$ and setting it to zero:

$$\hat{\mathbf{s}} = -\left(\frac{\partial^2 D}{\partial \mathbf{s}^2}\Big|_{\mathbf{s}_0}\right)^{-1} \frac{\partial D}{\partial \mathbf{s}}\Big|_{\mathbf{s}_0}$$

- Next, reject low contrast points and points that lie on the edge

SIFT: Keypoint Localization

- Reject points with strong edge response in one direction only
- Edge Elimination - How?
  - Similar to Harris corner detector!
  - SIFT instead uses Hessian
- Compute Hessian of D (principal curvature)

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

α : largest eigenvalue(λ_{max})

β : smallest eigenvalue(λ_{min})

$$Tr(H) = D_{xx} + D_{yy} = \alpha + \beta$$

$$Det(H) = D_{xx}D_{yy} - D_{xy}^2 = \alpha\beta$$

- Evaluate ratio

$$\frac{Tr(H)^2}{Det(H)} = \frac{(\alpha+\beta)^2}{\alpha\beta} = \frac{(r\beta+\beta)^2}{r\beta^2}$$

$$\frac{Tr(H)^2}{Det(H)} = \frac{(r+1)^2}{r} \text{ where } r = \frac{\alpha}{\beta}$$

- This quantity is minimum when r = 1 (eigenvalues are equal)
- Reject keypoint if: $\frac{Tr(H)^2}{Det(H)} >$ a threshold (Original SIFT uses r = 10)

SIFT: Orientation Estimation

- Why? To achieve rotation invariance
- Use scale of point to choose correct image:

$$\hat{I}(x,y) \quad G(x,y,\sigma) * I(x,y)$$

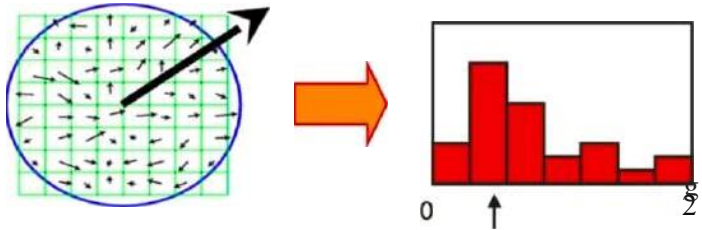
- Compute gradient magnitude and orientation using finite differences:

$$m(x,y) \quad \sqrt{(\hat{I}(x+1,y) - \hat{I}(x-1,y))^2 + (\hat{I}(x,y+1) - \hat{I}(x,y-1))^2}$$

$$\theta(x,y) = \tan^{-1}\left(\frac{(\hat{I}(x,y+1) - \hat{I}(x,y-1))}{(\hat{I}(x+1,y) - \hat{I}(x-1,y))}\right)$$

SIFT: Orientation Estimation

- Create histogram of gradient directions, within a region around the keypoint, at selected scale:



  - Histogram entries are weighted by:
    - gradient magnitude, and
    - a Gaussian function with σ equal to 1.5 times scale of the keypoint
  - Select the peak as direction of keypoint
  - Introduce additional key points at same location if another peak is within 80% of max peak of histogram with different direction

Credit: Svetlana Lazebnik, UIUC

SIFT



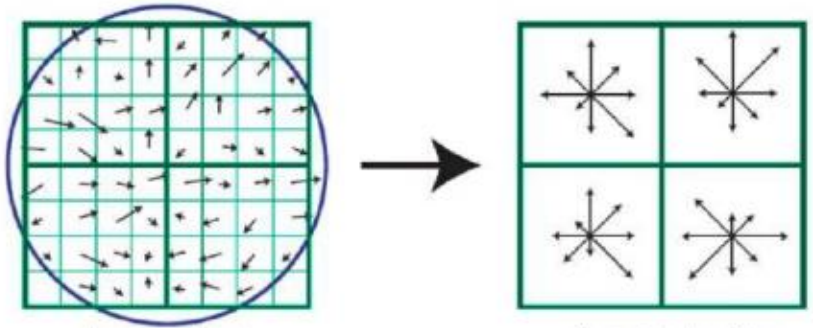


From 729 keypoints to 536 keypoints after testing ratio based on Hessian

Credit: Mubarak Shah, University of Central Florida

SIFT: KeyPoint Descriptor

- Compute gradient at each pixel in a 16×16 window around the detected keypoint, using the appropriate leve as detected.



SIFT: KeyPoint Descriptor

- Compute gradient at each pixel in a 16×16 window around the detected keypoint, using the appropriate leve as detected.

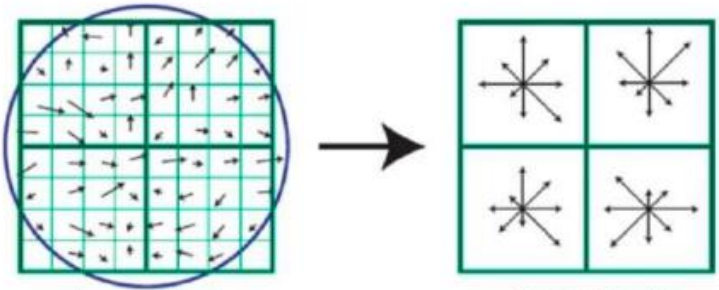


Image gradients

Keypoint descriptor

- Downweight gradients by a Gaussian fall-off function (blue circle) to reduce the influence of gradients far from the center.
- In each 4×4 quadrant, compute a gradient orientation histogram using 8 orientation histogram bins.

SIFT: KeyPoint Descriptor

- The resulting 128 non-negative values form a raw version of the SIFT descriptor vector.
- To reduce the effects of contrast or gain (additive variations are already removed by the gradient), the 128-D vector is normalized to unit length.
- To further make the descriptor robust to other photometric variations, values are clipped to 0.2 and the resulting vector is once again renormalized to unit length.

Credit. Riquel Urtasun, Szeliski

SIFT

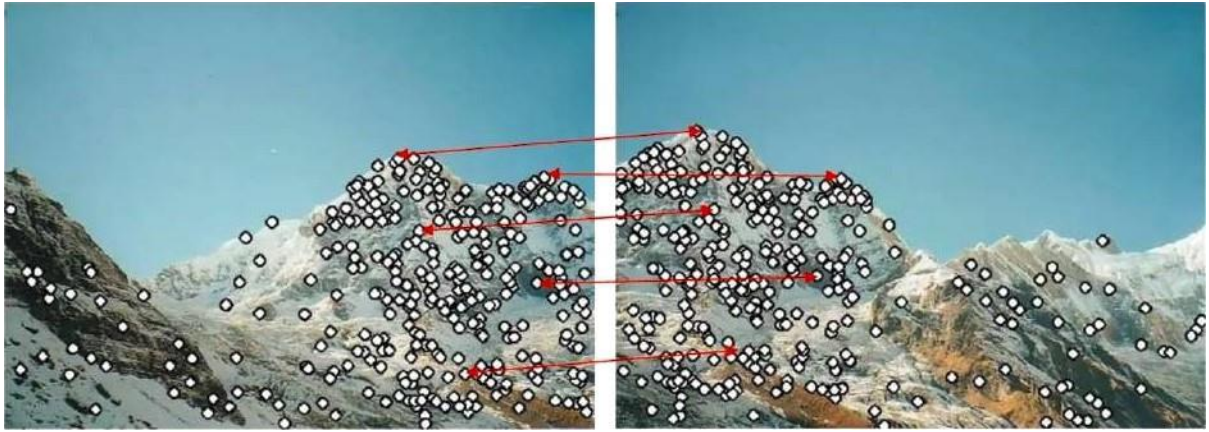
- Extraordinarily robust feature detection
- Changes in viewpoint: up to about 60 degree out of plane rotation
- Changes in illumination: sometimes even day vs night (below)
- Fast and efficient —can run in real-time





Credit. Raquel Urtasun, Szeliski

SIFT Application: Image Stitching



- Detect feature points in both images.
- Find corresponding pairs of feature points.